

开搞：搭建实验环境

说了这么多，现在该动手了。Make your hands dirty!

方便起见，可以先创建一个文件夹，作为你安装所有和 `riscv` 有关的软件的路径(例如 `/opt/riscv`)就行。后面安装的各个项目最好也放在上面的的路径里面。

最小的软件开发环境需要：能够编译程序，能够运行程序。开发操作系统这样的系统软件也不例外。

编译器

我们使用的计算机都是基于x86架构的。如何把程序编译到riscv64架构的汇编呢？这需要我们使用“目标语言为riscv64机器码的编译器”，在我们的电脑上进行**交叉编译**。

放心，这里不需要你自己写编译器。我们使用现有的riscv-gcc编译器即可。从<https://github.com/riscv/riscv-gcc> clone下来，然后在x86架构上编译riscv-gcc编译器为可执行的x86程序，就可以运行它，来把你的程序源代码编译成riscv架构的可执行文件了。这有点像绕口令，但只要有一点编译原理的基础就可以理解。不过，这个riscv-gcc仓库很大，而且自己编译工具链总是一件麻烦的事。

其实，没必要那么麻烦，我们大可以使用别人已经编译好的编译器的可执行文件，也就是所谓的**预编译（prebuilt）** 工具链，下载下来，放在你喜欢的地方（比如之前定义的 `$RISCV`），配好路径（把编译器的位置加到系统的 **PATH** 环境变量里），就能在终端使用了。我们推荐使用sifive公司提供的预编译工具链，进入<https://github.com/sifive/freedom-tools/releases>，找到并且下载适合你的操作系统的版本即可。（注意，如果你是wsl, 需要下载适合ubuntu版本的编译器）将安装包下的bin 添加到 `bashrc`当中。

对于编辑器的选择，**Ubuntu虚拟机**中，可以使用较为简单的图形化的gedit作为编辑器。

首先利用gedit进入`~/.bashrc`文档，运行以下命令：

```
gedit ~/.bashrc
```

在bashrc的最后添加路径

```
export RISCV=PATH_TO_INSTALL（你RISCV预编译链下载的路径）
export PATH=$RISCV/bin:$PATH
```

路径添加好了，点击保存并关闭~/.bashrc。

WSL中无法使用图形化编辑器，推荐使用nano编辑器。

首先利用在终端输入以下命令编辑 ~/.bashrc 文件：

```
nano ~/.bashrc
```

在 nano 中，按下 Ctrl + _，然后按下 Ctrl + V，这会将光标直接跳到文件的最后一行。

在文件的最后一行，输入修改内容。你需要添加以下两行来设置路径：

```
export RISCVC=PATH_TO_INSTALL (你RISCVC预编译链下载的路径)
export PATH=$RISCVC/bin:$PATH
```

输入完成后，按下 Ctrl + O 来保存文件，系统会提示你确认文件名，直接按 Enter 键确认保存。保存文件后，按下 Ctrl + X 退出 nano。

ps:使用**vscode**打开虚拟机/wsl的根目录，找到.bashrc文件并添加以上内容，可以实现同样的效果。

不管是虚拟机还是WSL，注意配置现在**还没有生效**！需要使用source命令使其生效：

```
source ~/.bashrc
```

配置好后，在终端输入 riscv64-unknown-elf-gcc -v 查看安装的gcc版本, 如果输出一大堆东西且最后一行有 gcc version 某个数字.某个数字.某个数字 ，说明gcc配置成功，否则需要检查一下哪里做错了，比如环境变量**PATH**配置是否正确。一般需要把一个形如bin 的目录加到**PATH**里。

须知

环境变量是操作系统中用来存储系统配置和运行参数的一种方式。它们通常用于存储关于用户、系统、运行环境等的信息，可以在操作系统的各个进程中共享和使用。例如，常见的环境变量有 PATH、HOME、USER 等。

那么为什么需要环境变量呢？

环境变量让我们可以在命令行中使用方便的别名或快捷方式，而不必每次都输入完整的路径。例如，系统会根据 PATH 环境变量来查找程序的位置。当你在终端输入一个命令时，操作系统会根据 PATH 中列出的路径去查找对应的可执行文件。

在这次实验中，我们创建了一个名为 RISCV 的环境变量，它用于指定你安装 RISC-V 相关软件的位置。这是为了让系统能够知道 RISC-V 工具链（比如编译器、调试工具等）在哪里，从而可以在终端中方便地执行相关命令。比如你在终端输入

`$RISCV=/opt/riscv` 时，操作系统就知道 RISCV 指定的路径在哪，它会根据这个路径找到所需要的工具，避免你每次都要手动输入完整路径。

总结来说，设置 RISCV 环境变量后，你可以随时在任何地方使用它来访问和执行 RISC-V 相关的软件工具，而不需要关心它们的具体位置。这种方法能够让开发过程更加高效和清晰。

模拟器

如何运行 riscv64 的代码？我们当然可以给大家每个人发一块 riscv64 架构处理器的开发板，再给大家一人一根 JTAG 线，让大家把程序烧写到上面去跑，然后各凭本事 debug（手动狗头）。但这种方式太笨重了，开发效率也低。更方便的方式是使用 **模拟器（emulator）**。

所谓模拟器，就是在 x86 架构的计算机上，用软件 **模拟出一台 riscv64 架构的计算机**，从而能够运行 riscv64 的目标代码。它的好处是，你不需要真正买一块开发板，也能在本机完成 OS 的开发与调试。

模拟器的原理

一台真实的计算机通常由 **CPU、内存、外设（硬盘、串口、网卡等）** 组成。模拟器的作用就是用软件去“假装”这一切：

对于 CPU 模拟，模拟器能够解析 riscv64 指令，然后通过解释或动态翻译的方式在 x86 CPU 上执行。换句话说，你写的 RISC-V 程序的每一条指令，都会在模拟器里被翻译成宿主机能理解的动作。

对于内存模拟，模拟器会在宿主机的内存里开辟一块区域，把它当作“riscv64 的物理内存”。当 riscv 程序访问某个地址时，模拟器就会把这个访问转到对应的宿主机内存地址上。

对于外设模拟，硬盘、串口、网卡等外设的真实硬件中通常通过寄存器和中断与 CPU 交互。模拟器会虚拟出这些寄存器接口，拦截指令并转化为宿主机上的操作。比如：模拟的“串口”输出会映射到你的终端。串口（Serial Port）是一种最简单的计算机输入输出接口，它通过一根数据线一位一位地传输信息，像 QEMU 这样的模拟器会虚拟出一块串口设备，把内核的串口输出直接映射到宿主机终端；模拟的“硬盘”其实对应宿主机上的一个镜像文件。在 QEMU 这种模拟器里，它并没有真正的 RISC-V 硬盘，而是用宿主机上的一个文件来“假装”是硬盘。这个文件通常叫“镜像文件”（image），它里面存储的就是硬盘的原始字节内容。模拟器会拦截客体系统发出的“硬盘读写请求”，把它转换成对镜像文件的读写；模拟的“时钟”则是用宿主机的计时器来实现。

简而言之：**模拟器就是一台用软件写出来的“假计算机”**。只要模拟得足够逼真，我们就能像在真实硬件上一样开发和调试操作系统。

下面我们从 [rCore tutorial](#) 抄写了一段 qemu 安装的教程。

安装模拟器 Qemu

如果你在使用 Linux (Ubuntu)，需要到 Qemu 官方网站下载源码并自行编译，因为 Ubuntu 自带的软件包管理器 apt 中的 Qemu 的版本过低无法使用。参考命令如下：

```
$ wget https://download.qemu.org/qemu-4.1.1.tar.xz
$ tar xvJf qemu-4.1.1.tar.xz
$ cd qemu-4.1.1
$ ./configure --target-list=riscv32-softmmu,riscv64-softmmu
$ make -j
$ sudo make install
```

可查看更详细的安装和使用命令。

如果你在使用 macOS，只需要 Homebrew 一个命令即可：

```
$ brew install qemu
```

最后确认一下 Qemu 已经安装好，且版本在 4.1.0 以上：

```
$ qemu-system-riscv64 --version
QEMU emulator version 4.1.1
Copyright (c) 2003-2019 Fabrice Bellard and the QEMU Project developers
```

使用 OpenSBI

新版 Qemu 中内置了 OpenSBI 固件 (firmware)，它主要负责在操作系统运行前的硬件初始化和加载操作系统的功能。我们使用以下命令尝试运行一下：

```
$ qemu-system-riscv64 \
  --machine virt \
  --nographic \
  --bios default

OpenSBI v0.4 (Jul  2 2019 11:53:53)

      ____
     /  __ \      /  ____|  _ \   _|
    | |  | |__    | |  |  | |  | |
    | |  | |  _  | |  |  | |  | |
    | |  | |  \_ /  \_ /  \_ /  \_ /
    | |  | |  | )  |  _/  | |  | )  | |
    \__ /  |  _/  \_ /  | |  | _/  | |
        | |
        | |
```

```
Platform Name      : QEMU Virt Machine
Platform HART Features : RV64ACDFIMSU
Platform Max HARTs   : 8
Current Hart        : 0
Firmware Base       : 0x80000000
Firmware Size       : 112 KB
Runtime SBI Version  : 0.1
```

```
PMP0: 0x0000000080000000-0x000000008001ffff (A)
PMP1: 0x0000000000000000-0xffffffffffff (A,R,W,X)
```

可以看到我们已经在 `qemu-system-riscv64` 模拟的 `virt machine` 硬件上将 `OpenSBI` 这个固件跑起来了。Qemu 可以使用 `Ctrl+a` 再按下 `x` 退出（注意要松开 `Ctrl` 再单独按 `x`）。

如果无法正常使用 Qemu，可以尝试下面这个命令。

```
$ sudo sysctl vm.overcommit_memory=1
```

扩展

如果对 OpenSBI 的内部实现感兴趣，可以看看[RISCV OpenSBI Deep_Dive 介绍文档](#)。